Trees

COMP7507
Visualization & Visual Analytics



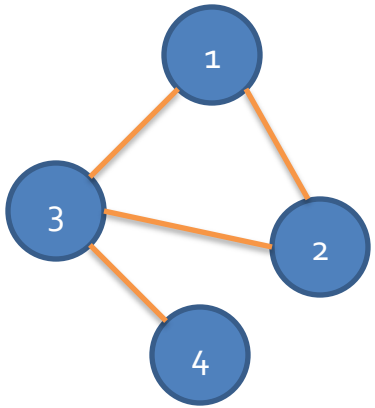
SCHOOL OF
COMPUTING &
DATA SCIENCE
The University of Hong Kong

Graphs

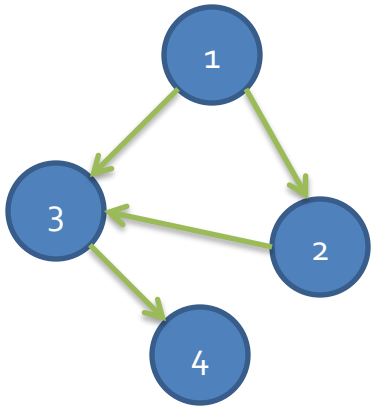
- **Graphs** are best for representing relational structures
  - Relational database
  - Object-oriented systems (class browsers)
  - Data structure (compiler data structure)
  - Real-time systems (state-transition diagrams)
  - Virtual reality (scene graphs)
  - Evolutionary trees, phylogenetic trees, genetic maps
  - Social networks, citation networks
  - Computer networks
  - World wide web
  - …

Graphs

- A graph **G=(V, E)** comprises a finite set **V** of vertices (or nodes) and a set **E** of edges (or lines), with each edge being an unordered pair (u, v) of vertices, $u, v \in V$.
- A **directed** graph consists of ordered pairs of vertices.



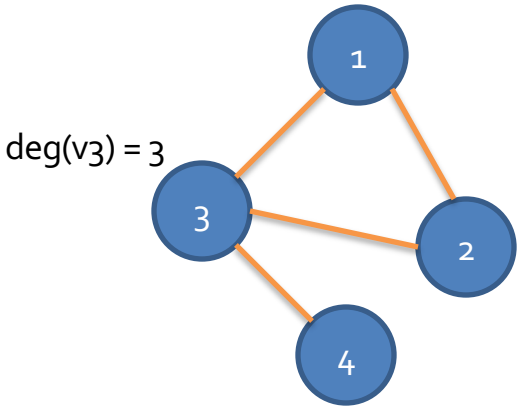
undirected graph



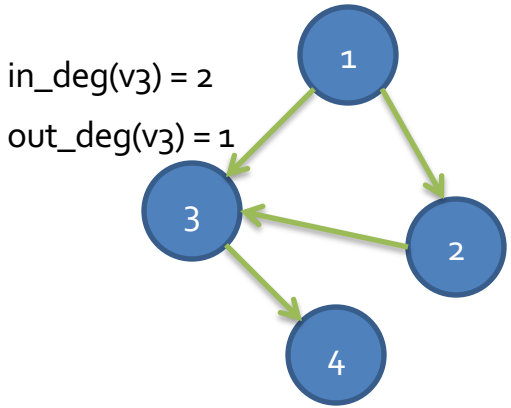
directed graph

Graphs

- The **degree** of a vertex
  = the number of edges incident to it
  = the number of neighboring vertices.



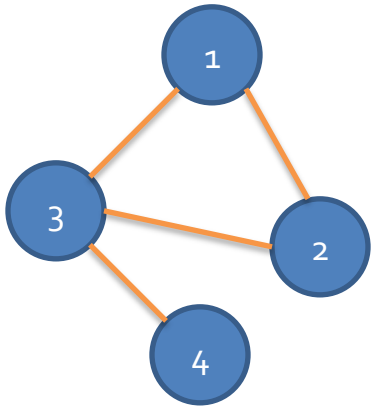
undirected graph



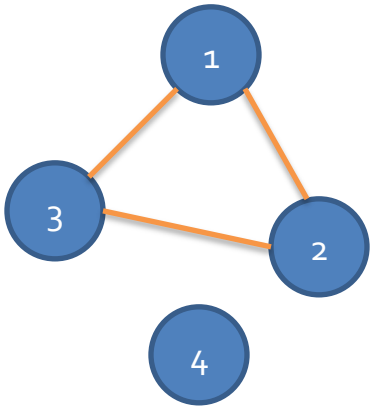
directed graph

Graphs

- A **path** is a sequence of edges connecting a sequence of vertices
- A graph is **connected** if there is a path between any two vertices.

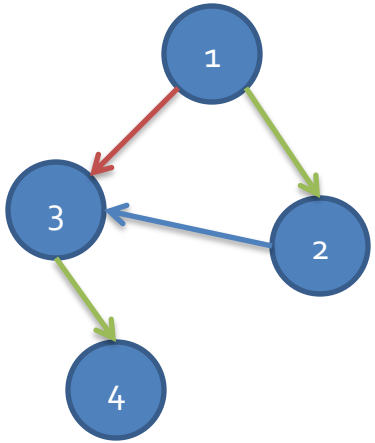


connected graph



disconnected graph

Graph Representation



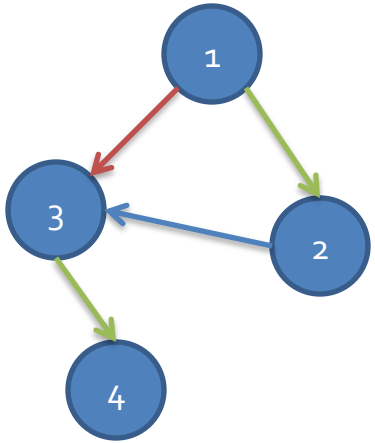
Adjacency Matrix

Source \ Target	1	2	3	4
1	0	2	1	0
2	0	0	3	0
3	0	0	0	2
4	0	0	0	0

Matrix entries storing edge weights or labels

- Random access to edges
- $O(n^2)$ space
- Not a compact storage, especially when the matrix is a sparse matrix (i.e., a graph with only a few edges)

Graph Representation



Adjacency List

1 -> 2, 3
2 -> 3
3 -> 4

List the edges for each vertex

Edge List

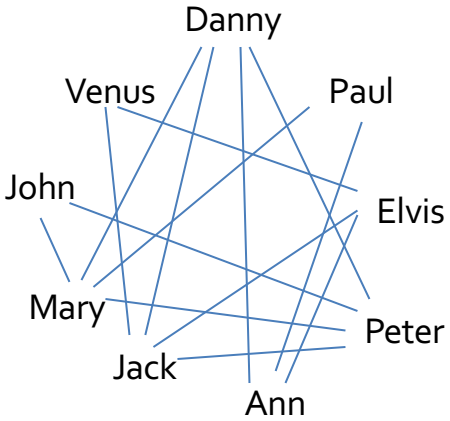
1, 2, **2**
1, 3, **1**
2, 3, **3**
3, 4, **2**

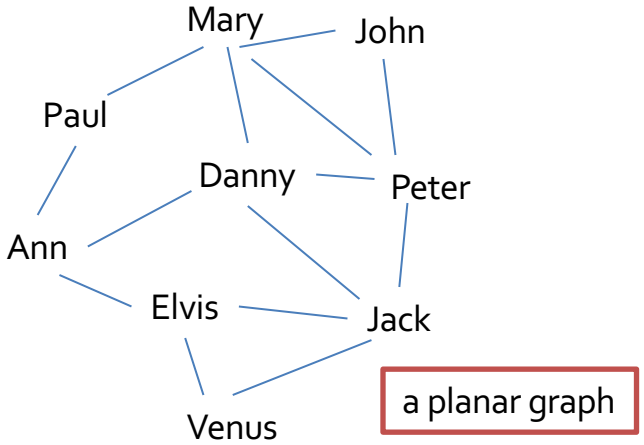
Enumerates all edges

- Storage proportional to number of edges
- However, only support sequential access to edges

Graph Drawing

- There are many different ways of drawing a graph.





- Whether a drawing is good or not is subjective and depends on perception.

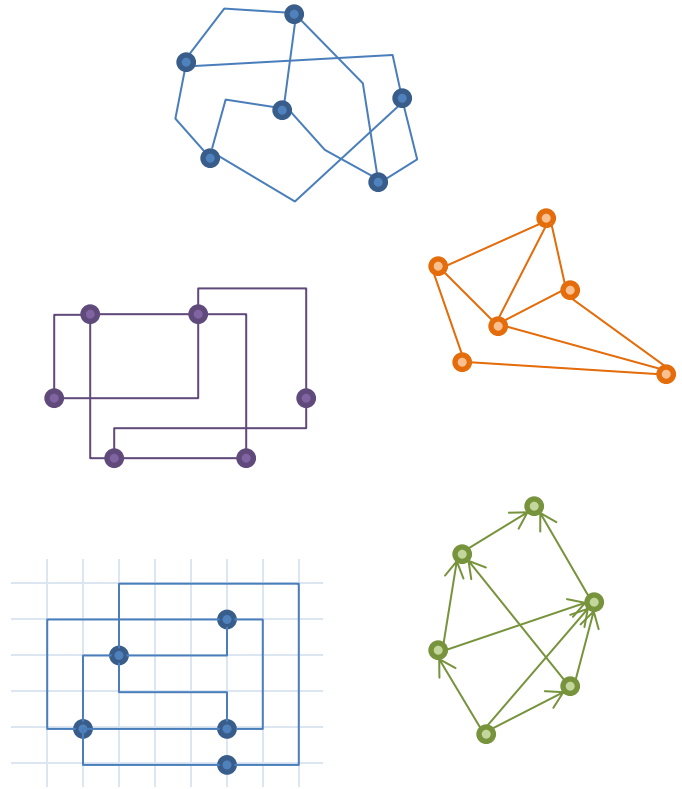
Graph Drawing

Requirements

- Drawing conventions
  - Rules specific to certain applications / professions
- Aesthetics
  - Readability, layout
- Constraints

Drawing Conventions

- Straight-line drawing
- Polyline drawing
- Orthogonal drawing
- Planar drawing
- Grid drawing
- Upward drawing

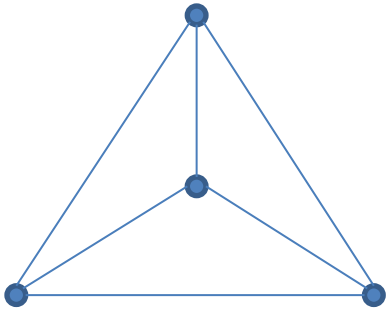


Aesthetics

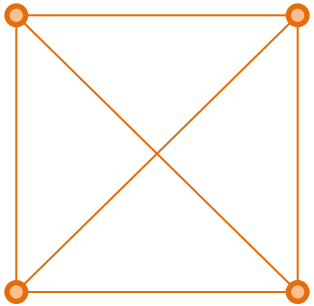
- Aims to characterize **readability** with general optimization goals:
  - Minimize
    - # of crossings
    - Total area
    - Total edge length
    - Maximum edge length
    - Total number of bends (in orthogonal drawings)
  - Maximize
    - Angular resolution (smallest angles between two edges)
    - Maximum display of symmetry
  - Maintain aspect ratio of the entire drawing

Aesthetics

- In general, aesthetic criteria are in conflicts.



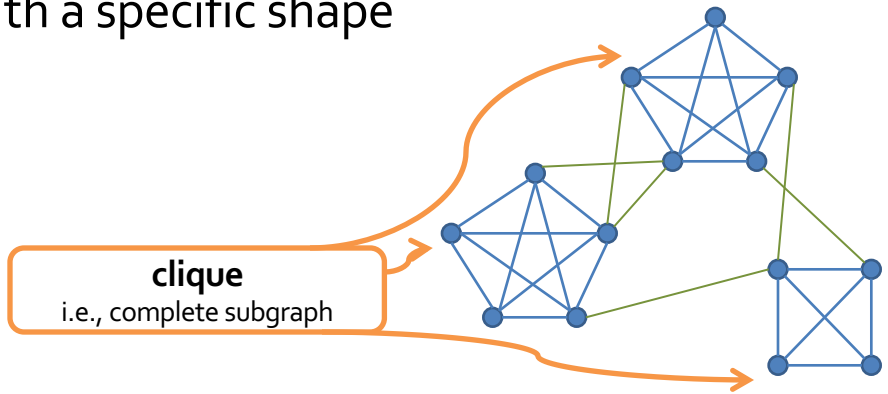
No edge crossing,
but asymmetric



Symmetric,
but with edge crossing

Constraints

- Enhance readability by adding knowledge about the **semantics** of the graph
  - Place a vertex close to the middle
  - Place a vertex on the external boundary
  - Place a subset of vertices close together
  - Draw a subgraph with a specific shape



Graph Drawing

Key issues

- Graph size
  - need filtering, clustering?
- Predictability
  - similar drawing for the same graph every time?
- Time complexity
  - Is real-time interaction possible?

Trees

Trees

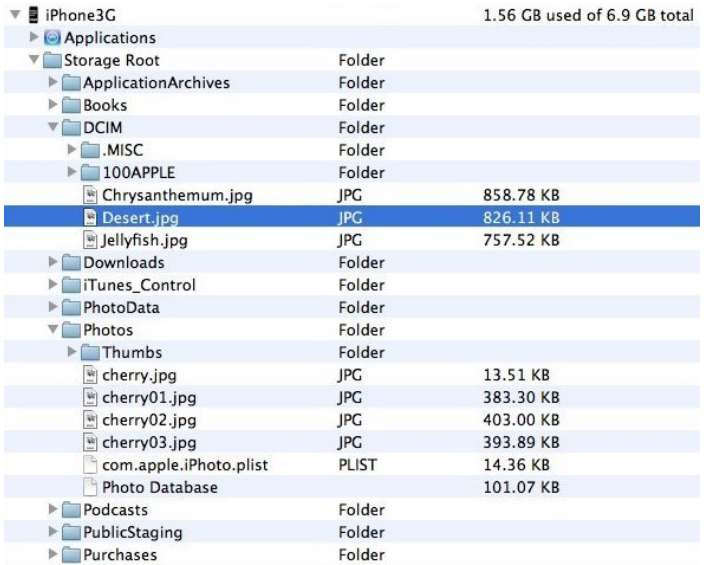
- A **tree** is a directed acyclic graph:
  - Exactly one unique vertex called the root with no parents
  - Every vertex except the root has a parent
  - There is a path from the root to each vertex
- Trees are good for representing:
  - Hierarchies (file systems, web sites, organization charts)
  - Branching Processes (family lineage, evolution)
  - Decision processes (search trees, game trees, decision trees)

Tree Layouts

- Indentation
  - Tree depth is encoded by indentation
- Node-Link Diagrams
  - Nodes connected by lines/curves
- Layered Diagrams
  - Hierarchical structure represented by layering, adjacency or alignment
- Enclosure/Containment Diagrams
  - Hierarchical structure represented by enclosure

- In general, tree layout can be done efficiently in O(n) or O(n log n) time

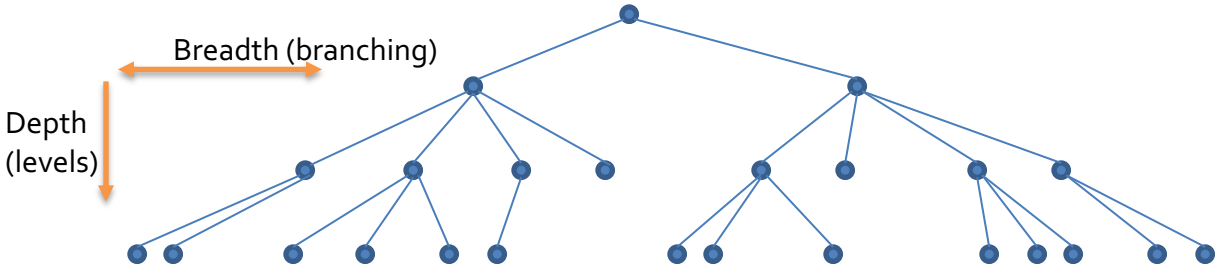
Indentation



iPhone3G		1.56 GB used of 6.9 GB total
Applications		
Storage Root	Folder	
ApplicationArchives	Folder	
Books	Folder	
DCIM	Folder	
.MISC	Folder	
100APPLE	Folder	
Chrysanthemum.jpg	JPG	858.78 KB
Desert.jpg	JPG	826.11 KB
Jellyfish.jpg	JPG	757.52 KB
Downloads	Folder	
iTunes_Control	Folder	
PhotoData	Folder	
Photos	Folder	
Thumbs	Folder	
cherry.jpg	JPG	13.51 KB
cherry01.jpg	JPG	383.30 KB
cherry02.jpg	JPG	403.00 KB
cherry03.jpg	JPG	393.89 KB
com.apple.iPhoto.plist	PLIST	14.36 KB
Photo Database		101.07 KB
Podcasts	Folder	
PublicStaging	Folder	
Purchases	Folder	

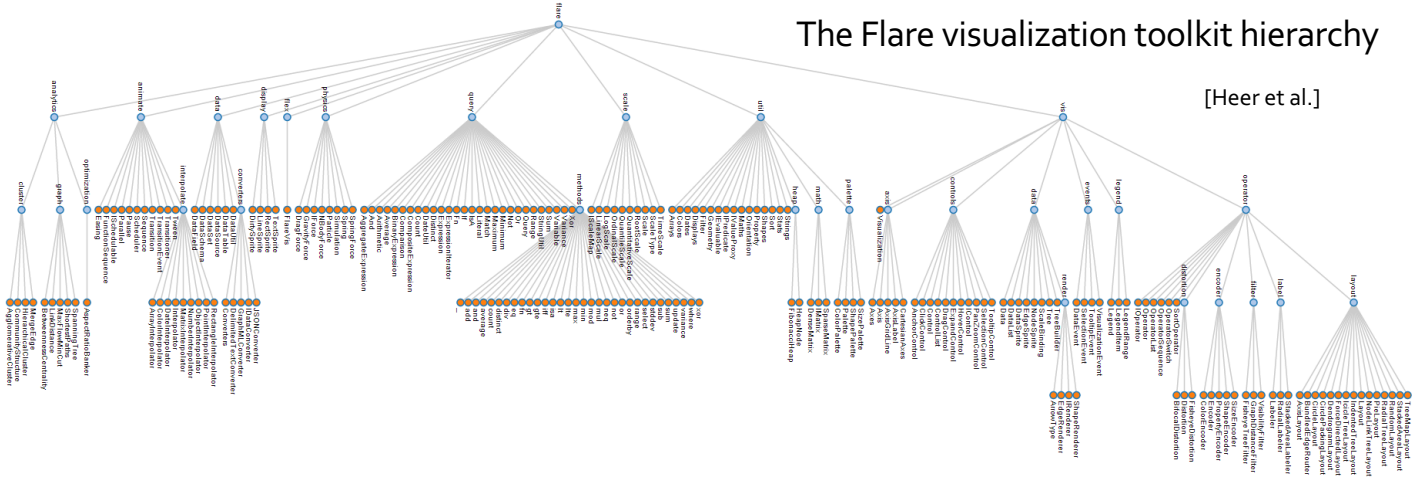
- Items are listed in rows
- Indentation is used to show parent/child relationship
- Multivariate data can be displayed in columns
- Takes up excessive vertical space

Node-Link Diagrams



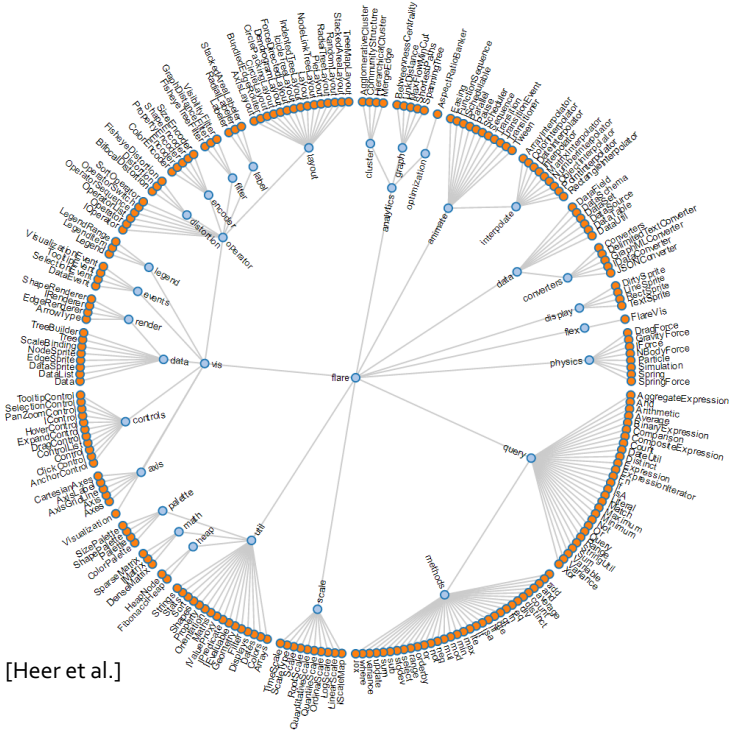
- Objects represented as nodes are distributed in space; relationships are represented by straight or curved lines connecting nodes
- Breadth and depth of a tree are layout in different directions/dimensions in 2D
- Problem: breadth may grow exponentially when branching factor is large

Reingold-Tilford Algorithm



- A classical layout in which children nodes are positioned under their common parent
- Same drawing for isometric subtrees and no edge crossing
- Leave nodes are at different levels

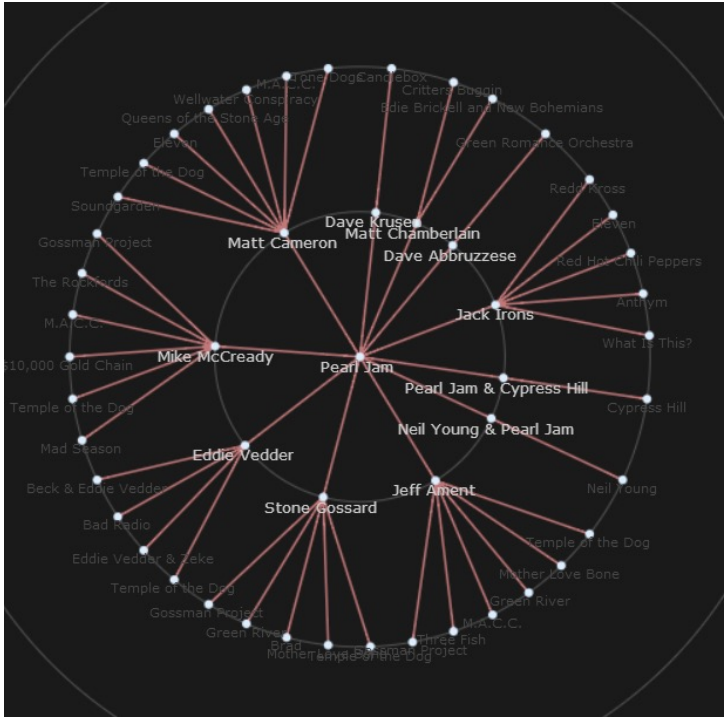
Radial Layout



The Flare visualization toolkit hierarchy

- Node-link diagram but in polar co-ordinates
- Leaf nodes are all at the circumference (dendrogram)
- Root node at the center
- Visually more pleasing and spatially more efficient

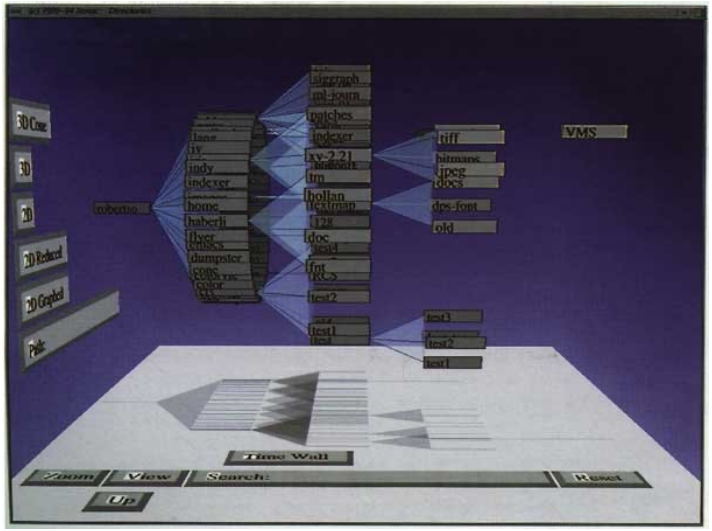
Radial Layout



An animated radial tree layout with root change

[http://philogb.github.io/jit/static/v20/Jit/Examples/RGraph/example1.html]

Cone Trees

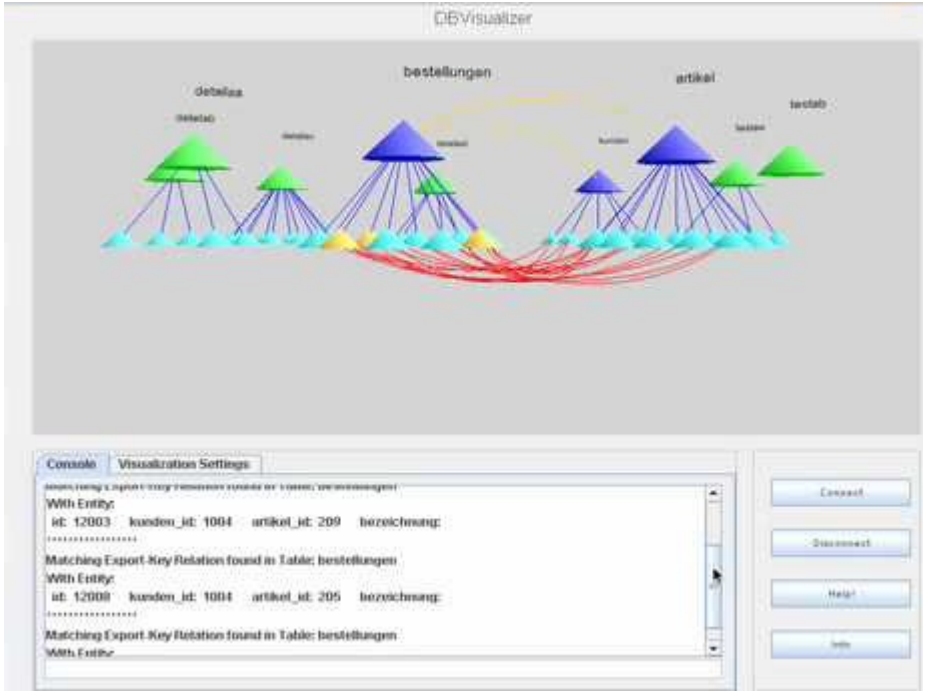


Robertson et al., "*Cone Trees: Animated 3D Visualizations of Hierarchical Information*", CHI'91.

- Make use of more space with 3D, i.e., an additional dimension
- Nodes at apex of a cone
- Children evenly spaced along the cone base

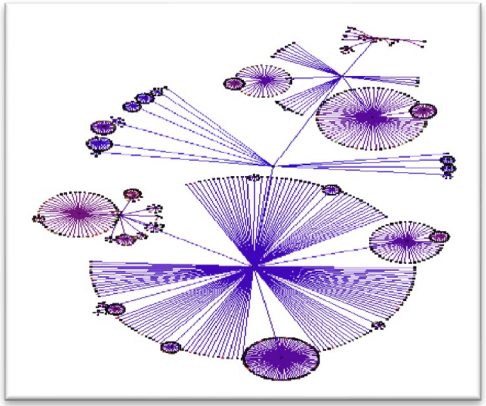
Cone Trees

- An example



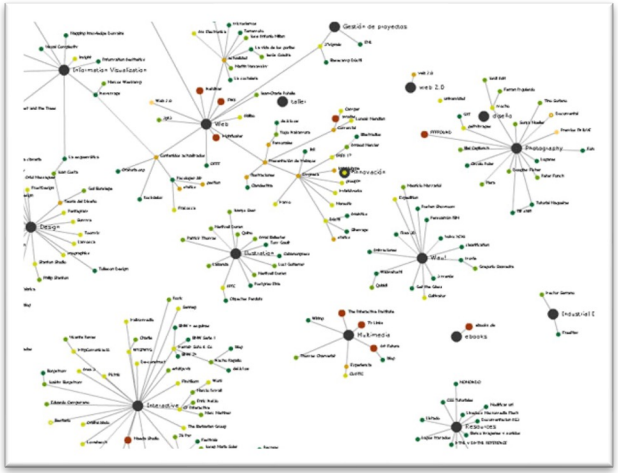
[https://www.youtube.com/watch?v=1eO1pgTVu-g]

Balloon Trees



Carriere and Kazman, "Research Report: Interacting with Huge Hierarchies: Beyond Cone Trees", *Proc. of Information Visualization '95.*

- Children are positioned around their parent
- Like flattening a cone tree, but without overlapping among circles.

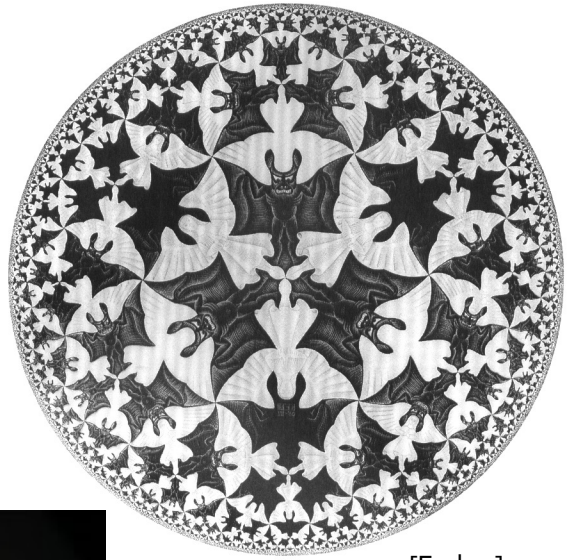


[http://www.webdesignerdepot.com/2009/06/50-great-examples-of-data-visualization/]

Problems of Node-Link Diagrams

- Tree breadth may grow exponentially
- Easily run out of space, even with space efficient layout
- Solution:
  - Filtering
  - Clustering / Aggregation
  - Interactions, e.g., focus + context, scroll & pan, zoom
  - Distortion (use different aspect ratios at different regions)

Hyperbolic Layout



[Escher]

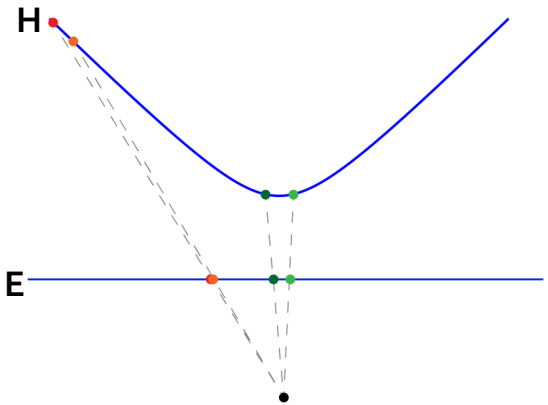
Can you picture that the creatures in Escher's drawing here are all of the same size?

Illuminating hyperbolic



Hyperbolic Layout

- A distorted view of a tree so that the region in focus has more space for layout

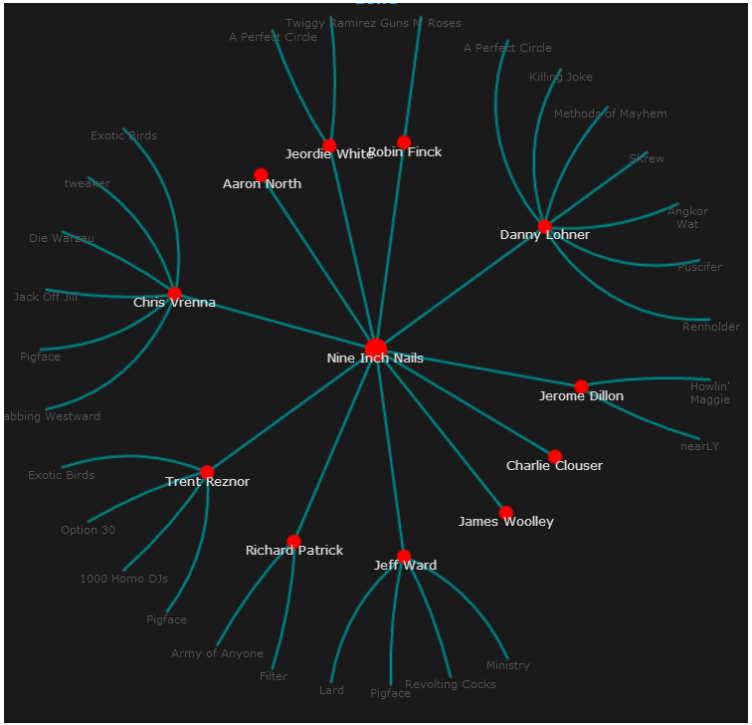


A shape of same size at different locations of the hyperbolic plane are projected to shapes of different sizes on the Euclidean plane

- Design a tree layout in the hyperbolic plane, then display the layout in the Euclidean plane

How may we change the focus of the tree layout?

Hyperbolic Layout



[http://philogb.github.io/jit/static/v20/Jit/Examples/Hypertree/example1.html]

Layered Diagrams

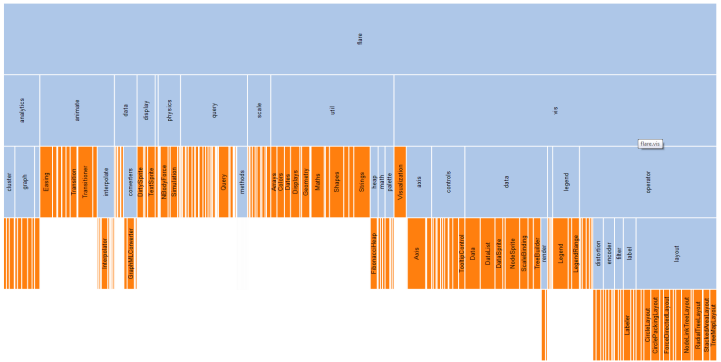
Space-filling variant of the node-link diagrams

Nodes are drawn as solid areas with lengths encoding an additional dimension

Relative position of nodes implies node relationships

Hierarchical structure represented by

- Layering
- Adjacency
- Alignment

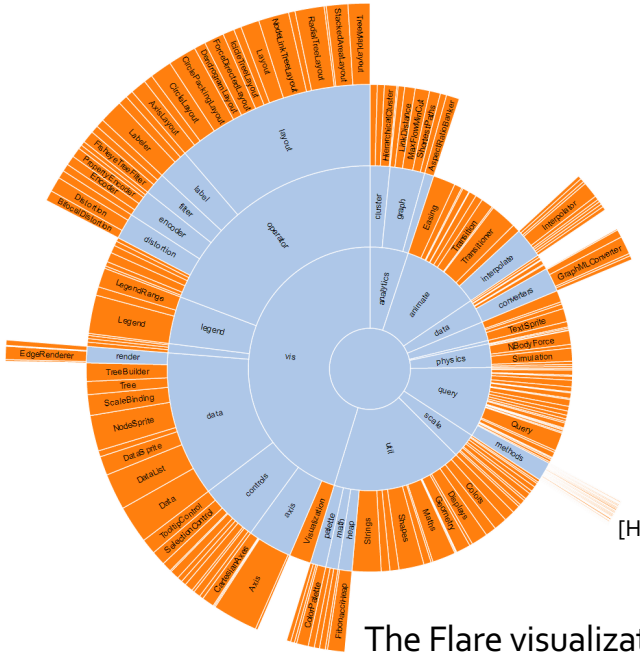


Icicle Layout

Layered Diagrams

Another example

Sunburst Layout



[Heer et al.]

The Flare visualization toolkit hierarchy

Enclosure/Containment Diagrams

Enclosure diagrams are space filling and makes maximal use of the display space

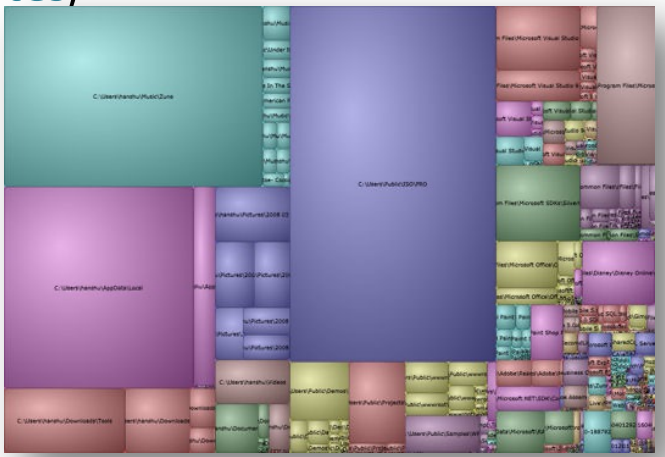
Relations are represented by enclosure or containment of shapes, e.g., rectangles / circles

Size of shapes can be used to show attributes, e.g., country population

Colors are used to show attributes or hierarchical relationships

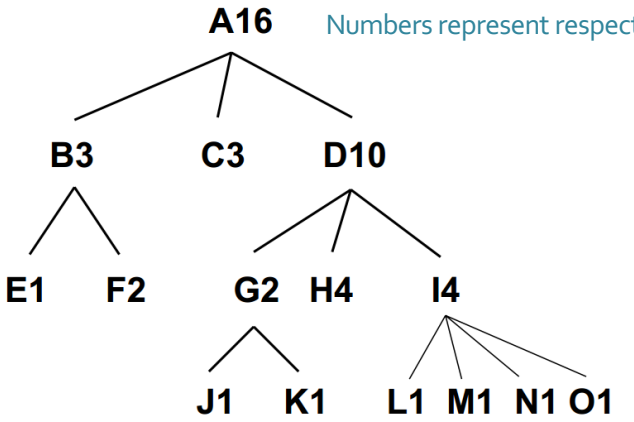
Examples:

- TreeMaps and its variants

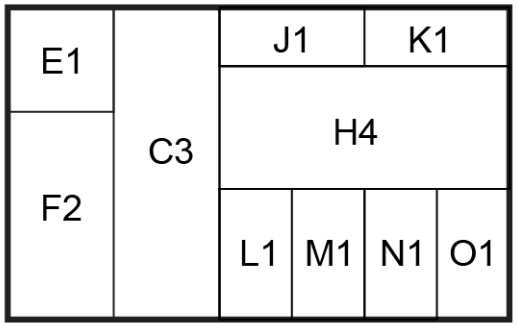


Treemaps

- A compact representation by subdividing an area into rectangles recursively
- The direction of subdivision alternates per level



Numbers represent respective data size

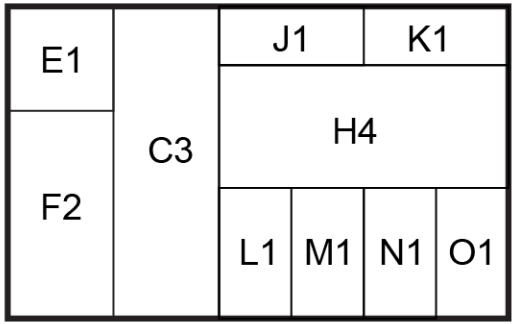


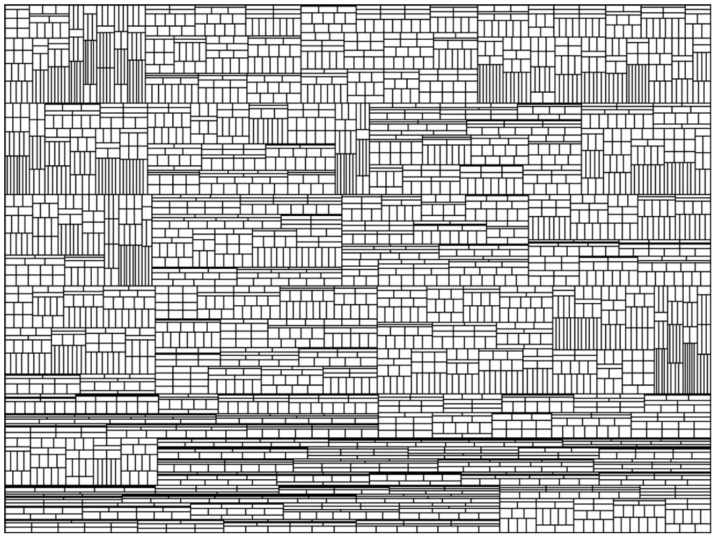
In this example, we subdivide horizontally at the first level

Johnson and Shneiderman, "Tree-Maps: a space-filling approach to the visualization of hierarchical information structures", VIS '91.

Treemaps

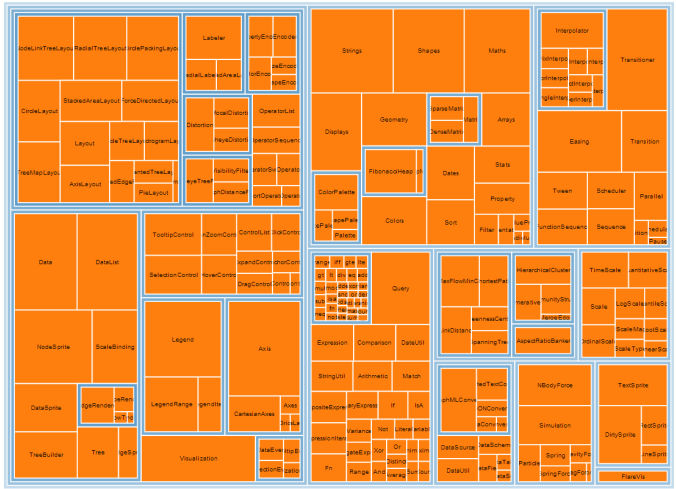
- Advantages:
  - Gives a single overview of a tree
  - Large or small nodes are easily identified
- Problems:
  - Difficult to perceive hierarchical structure





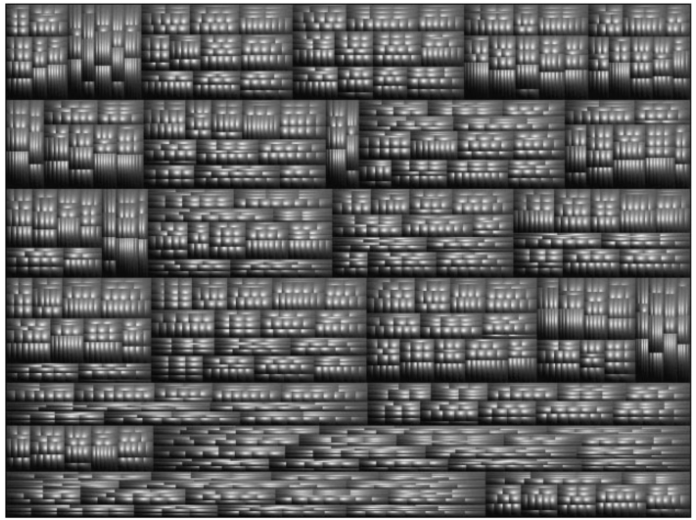
Treemaps

- To enhance hierarchical structure:



Use frame / padding

[Heer et al.]



Use shading (Cushion treemap)

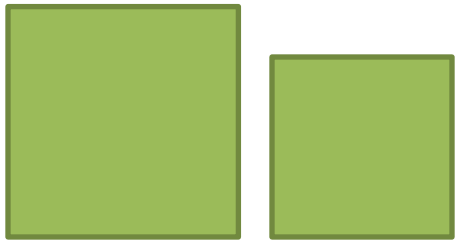
Van Wijk and van de Wetering, "Cushion Treemaps: Visualization of Hierarchical Information", INFOVIS '99.

Rectangles vs. Squares?

- Which is easier to compare, in terms of size?



Which one is bigger?



Which one is bigger?

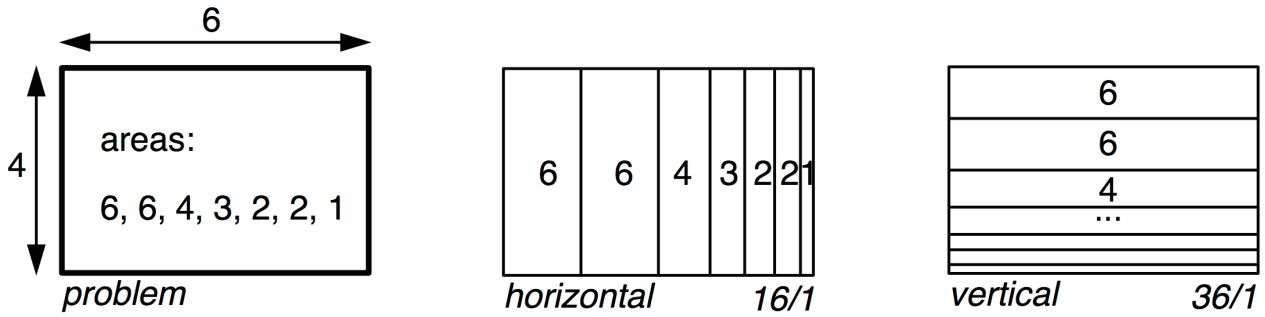
Squarified Treemaps

Aspect ratio of a rectangle is taken as max(height/width, width/height)

- Observation:
  - we can compare better the size of rectangles when their aspect ratios are similar.
  - subdividing area in a single dimension at each level (as is done in ordinary treemap) results in elongated rectangles.
- Improvement:
  - choose either horizontal and vertical split at each level
  - add a rectangle to a row only when this will improve the layout (i.e., aspect ratio)
- Adopt a greedy approach to process large rectangles first, in order to achieve better aspect ratios.
- Final treemap layout is not guaranteed to be optimal.

Squarified Treemaps

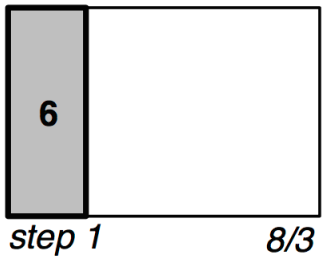
- An example
  - To layout a tree node with seven children whose sizes are 6, 6, 4, 3, 2, 2, 1 in a rectangle of size 6 x 4.



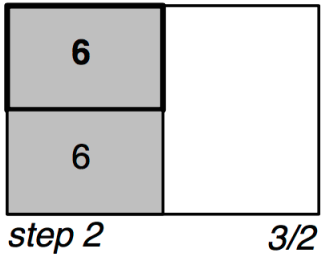
Ordinary treemap

[Bruls et al., " Squarified Treemaps", TCVG 2000]

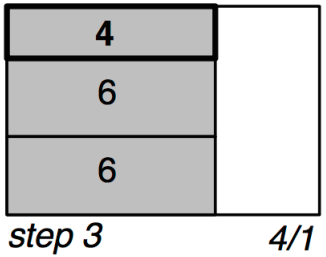
Squarified Treemaps



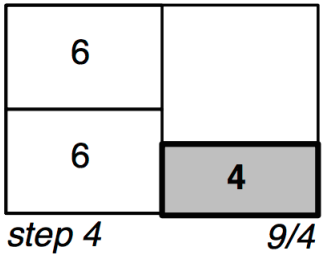
Subdivide horizontally, and add a new rect (starting from the largest one)



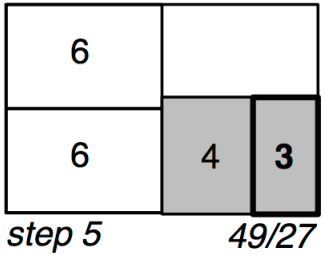
Add 2nd rect on top of 1st to improve the aspect ratio



Add 3rd rect on top and the aspect ratio worsen, hence discard layout



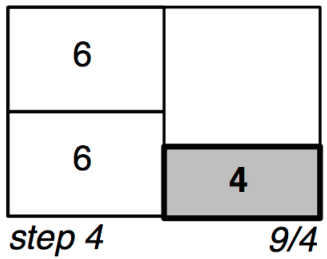
Subdivide remaining area vertically, and add a new rect



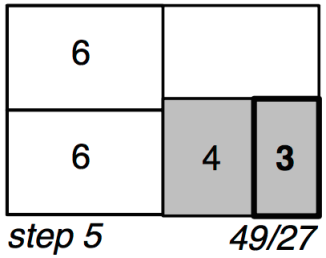
Add next rect to the right to improve the aspect ratio

Aspect ratio of a rectangle is taken as max(height/width, width/height)

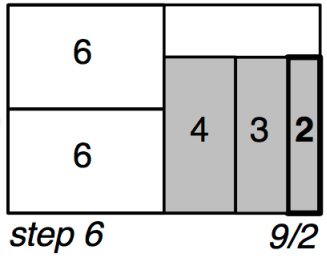
Squarified Treemaps



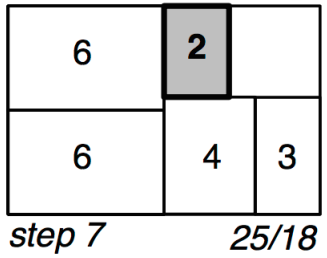
Subdivide remaining area vertically, and add a new rect



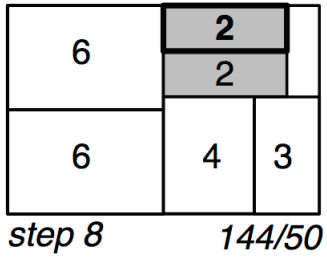
Add next rect to the right to improve the aspect ratio



Add next rect to the right and the aspect ratio worsen, hence discard layout

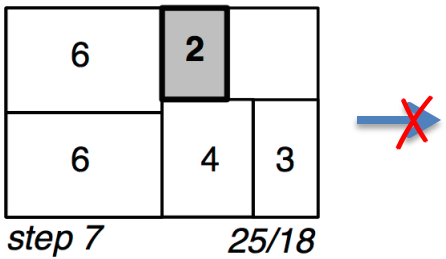


Subdivide remaining area vertically, and add a new rect

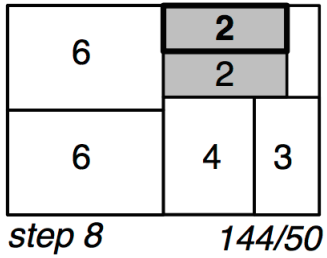


Add next rect to the top of previous and the aspect ratio worsen, hence discard layout

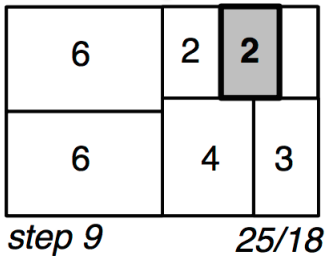
Squarified Treemaps



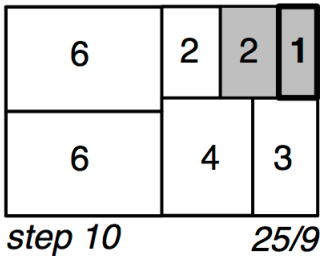
Subdivide remaining area vertically, and add a new rect



Add next rect to the top of previous and the aspect ratio worsen, hence discard layout

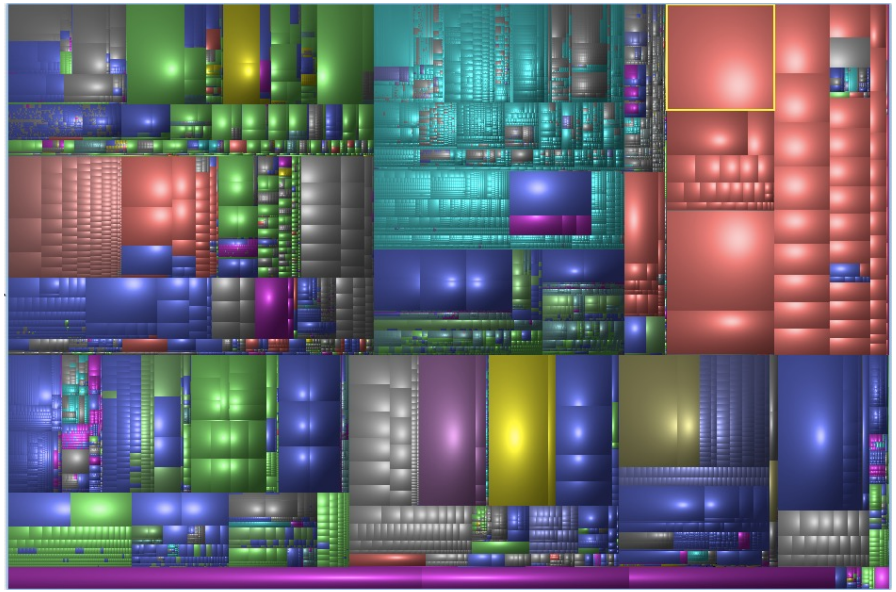


Subdivide remaining area vertically, and add a new rect



Add the last rect and done

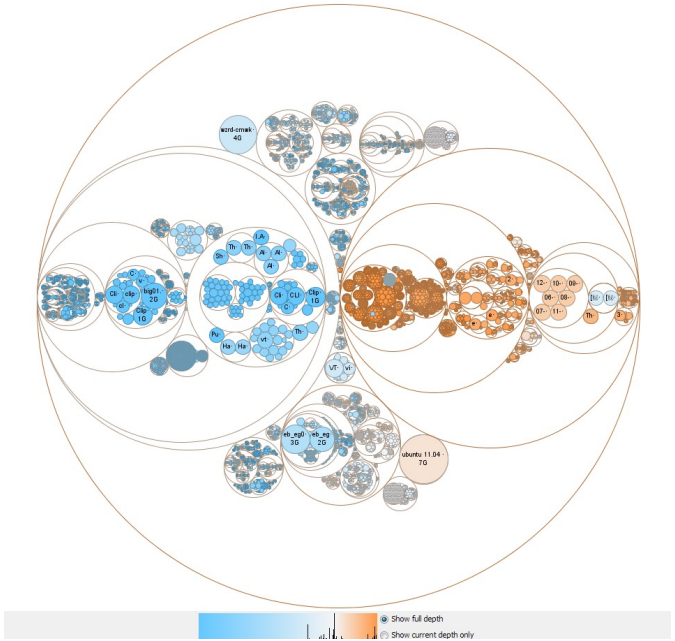
Squarified Treemaps



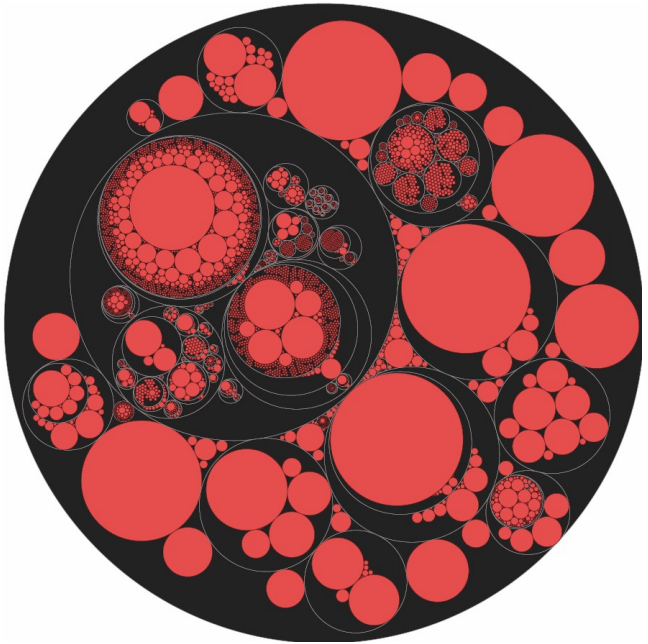
- Problems:
  - Change in dataset causes dramatic discontinuous change
  - Orders not preserved (Solution: Ordered Treemaps)

Circular Treemaps

- Use circles instead of rectangles



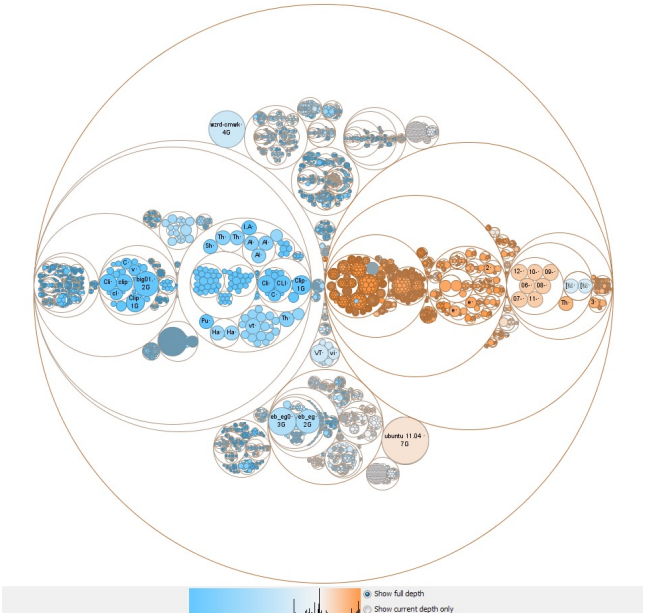
[Tree Visualizer]



[Pebbles by Wetzel]

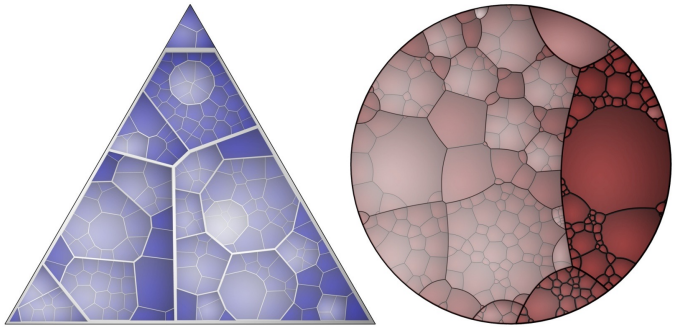
Circular Treemaps

- Advantages:
  - Easy to compare sizes
  - Hierarchical structure is clear
- Problems:
  - Not as space efficient



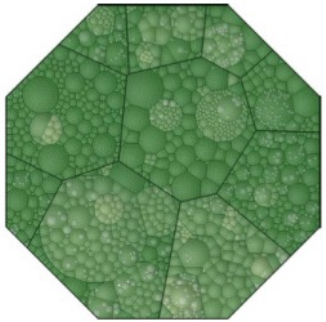
Voronoi Treemaps

- Use arbitrary polygons to fill up an arbitrary space
- Boundaries not just formed by vertical and horizontal lines, therefore easier to see hierarchical structure



[Balzer and Deussen, "Voronoi treemaps", InfoVis 2005.]

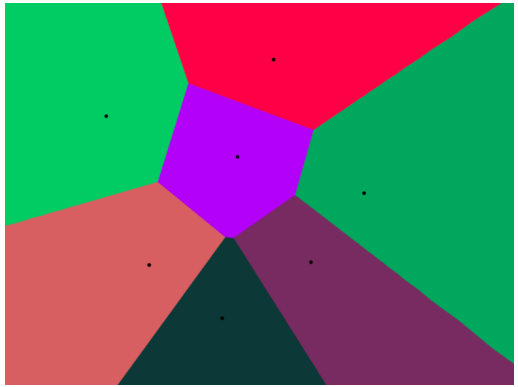




[Paul Murrel, University of Auckland]

Voronoi Treemaps

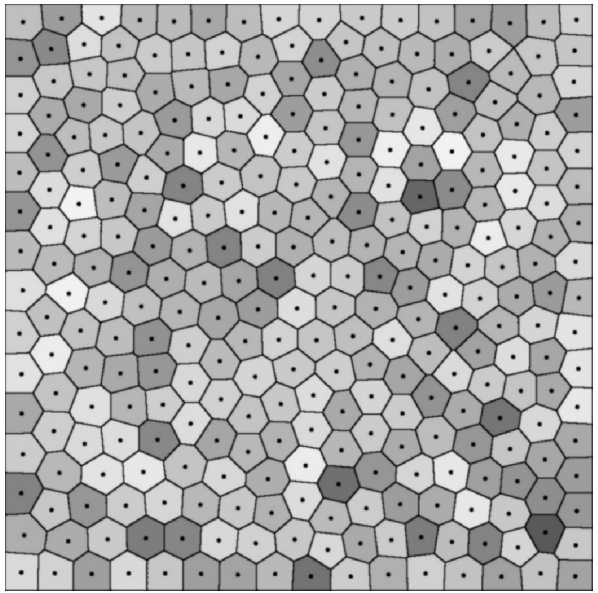
- Use the idea of **weighted centroidal Voronoi tessellation (CVT)** for generating the treemaps.
- Given a set of seed points, a Voronoi tessellation is a partition of a domain into cells (one for each seed point), so that every point within a cell is closer to its seed point than to other seed points.



[http://alexbeutel.com/webgl/voronoi.html]

Voronoi Treemaps

- **A centroidal Voronoi tessellation (CVT)** is a Voronoi tessellation in which all seed points are the centroid of their respective cell.



a CVT

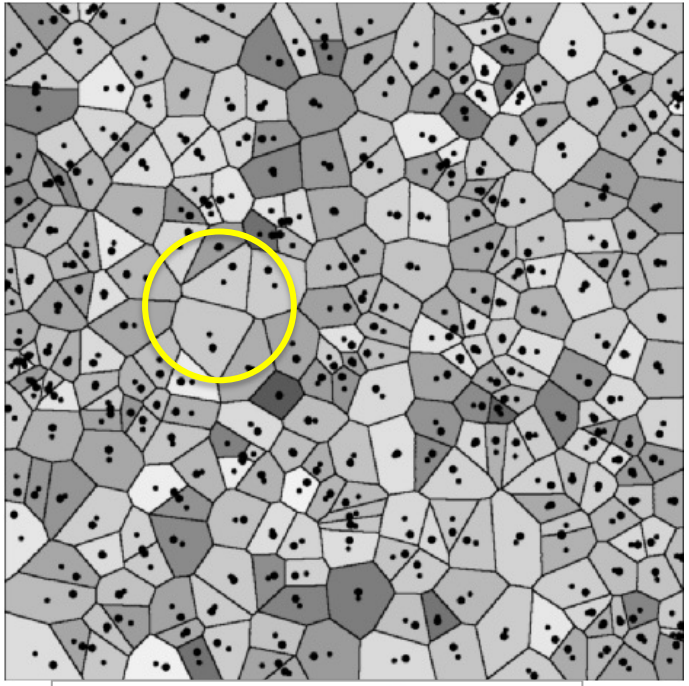
- Seed points are evenly distributed
- Cells are about the same size
- Weighted CVT: seed points are weighted so a seed point with higher weight attracts a larger neighbourhood

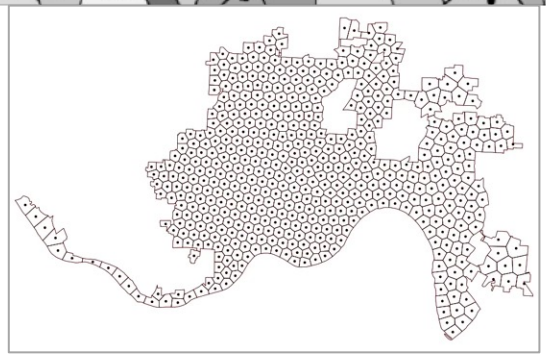
Voronoi Treemaps

How to compute a CVT for k seeds?

Lloyd's relaxation

- Determine initial positions for the k seeds
- Repeat the following relaxation step until convergence, i.e., when the site positions are fixed (ideal case)
  - Compute the Voronoi tessellation of the k sites
  - Move each site to the centroid of its Voronoi cell

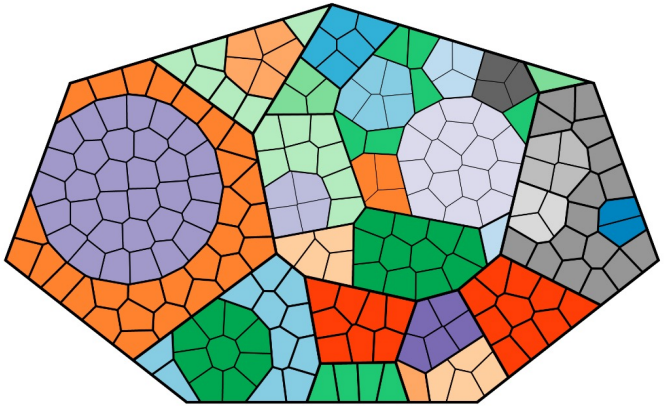




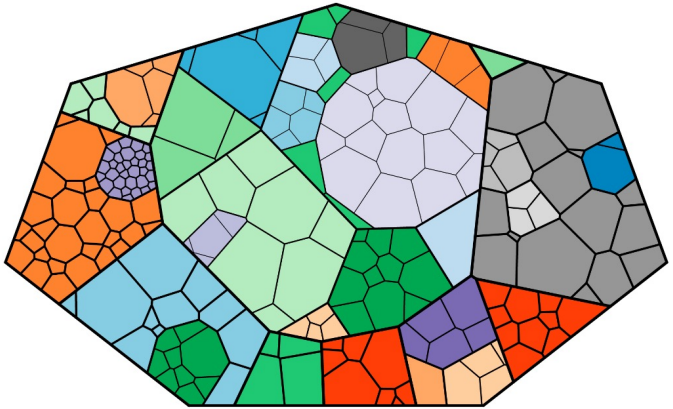
https://youtu.be/HYDSxgNT1zc

Voronoi Treemaps

[https://www.jasondavies.com/voronoi-treemap/]



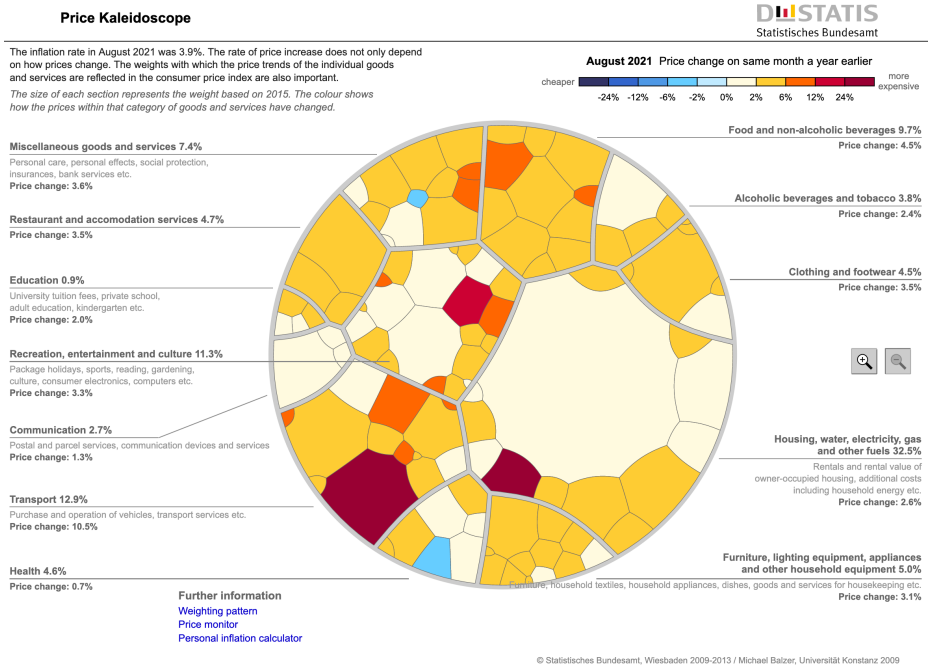
Cells of equal area



Cells of area in proportion to data values

- Problems:
  - Computation involves an iterative process, which can be inefficient

Voronoi Treemaps



[https://www.destatis.de/EN/Themes/Economy/Prices/Consumer-Price-Index/price-kaleidoscope-overview.html]

Reference

- Jeffrey Heer, Michael Bostock, and Vadim Ogievetsky. 2010. A tour through the visualization zoo. *Commun. ACM* 53, 6 (June 2010), 59-67. (http://hci.stanford.edu/jheer/files/zoo/)
- Matthew Ward, Georges Grinstein and Daniel Keim, "*Interactive Data Visualization: Foundations, Techniques, and Applications*", 2010 [Chapter 8]
- Isabel F. Cruz and Roberto Tamassia, "Graph Drawing Tutorial" (https://www.fmf.uni-lj.si/~zitnik/dmii/graph_drawing_tutorial.pdf)